Рекомендательная система фильмов

Программная инженерия. Третья часть задания

Выполнил: Лапаев Степан. Группа: 11-412

1 Задача работы

Нужно сделать программу, которая предлагает пользователю фильмы. По заданию нужны два классических способа и один или два простых. Также нужно сравнить результаты, описать недостатки и объяснить, как проверять пользу рекомендаций в настоящем сервисе.

Сделаны три способа: по жанрам, по похожим пользователям и по популярности. При запуске указываются номер пользователя, способ и число фильмов. Результат выводится в терминал и сохраняется в JSON. Уже оценённые пользователем фильмы в список не попадают.

Программа запускается локально через Python. NumPy используется для массивов, scikit-learn для сравнения фильмов и пользователей, PyTest для тестов. Сайт и API в этой части не создавались.

2 Данные

Выбран MovieLens 100K [1, 2]. В нём 100000 оценок от 943 пользователей для 1682 фильмов. Оценки идут от 1 до 5. Они собраны в 1997-1998 годах, поэтому по ним нельзя судить о вкусах современных зрителей.

В u.data находятся номер пользователя, номер фильма, оценка и время её выставления. В u.item записаны названия фильмов и их жанры. Названия жанров перечислены в u.genre. Личные сведения о пользователях не используются.

Данные разделены по времени: первые 80003 оценки используются для построения рекомендаций, остальные 19997 для проверки. Записи с одинаковым временем остаются вместе. Все проверочные оценки идут позже тех, на которых строится список. Программа не получает проверочные оценки заранее.

Для рекомендаций доступны 1616 фильмов с хотя бы одной оценкой в первой части данных. Из них убираются фильмы, которые выбранный пользователь уже оценил. 95 высоких проверочных оценок относятся к новым фильмам вне этого списка, поэтому они не участвуют в расчёте качества. Работа с новыми фильмами отдельно не проверялась.

В сравнении участвуют 287 пользователей, у которых есть проверочная оценка 4 или 5 для доступного фильма. У 95 из них есть история в первой части данных, у 192 её нет. Набор скачивается с сайта GroupLens. Папка data не включается в архив и репозиторий, поскольку условия набора ограничивают его распространение [2].

3 Способы рекомендаций

По жанрам (content). Программа смотрит на жанры фильмов, которым пользователь поставил 4 или 5, и ищет похожие фильмы. Жанры переводятся в числа с помощью TF-IDF: более редкие жанры получают больший вес [3]. Эти веса считаются только по доступным фильмам из первой части данных. Для пользователя усредняются числа его понравившихся фильмов. Затем косинусное сходство сравнивает полученный набор с каждым доступным фильмом [4].

Плюс этого способа: рекомендацию можно объяснить похожими жанрами. Минус: сюжет, актёры и режиссёр не учитываются. Поэтому разные фильмы с одинаковыми жанрами получают одинаковое число для сортировки.

По похожим пользователям (collaborative). Программа сравнивает списки фильмов, которым люди поставили 4 или 5. Берутся до 30 наиболее похожих пользователей. Их понравившиеся фильмы участвуют в составлении списка. Мнение более похожего человека имеет больший вес. Сам пользователь с собой не сравнивается.

Этот способ учитывает выбор других людей, а не только жанры. Но при малом числе оценок похожих людей может не найтись. Число рядом с фильмом служит для сортировки: это не прогноз оценки от 1 до 5. Оно считается как сумма голосов похожих пользователей с учётом их веса, делённая на сумму весов.

По популярности (popular). Для каждого фильма считается число оценок 4 и 5 в первой части данных. Чем больше таких оценок, тем выше фильм в списке. Например, 410 означает 410 высоких оценок. Средняя оценка здесь не используется.

Этот способ простой и подходит для нового пользователя. Но популярный фильм не обязательно понравится конкретному человеку. Если два других способа не могут составить личный список, программа переключается на популярность. Отсутствие истории обычно называют холодным стартом.

Во всех способах исключаются фильмы с любой уже известной оценкой пользователя, даже низкой. При равных результатах первым идёт фильм с меньшим номером. Оценки 1-3 пока используются только для исключения просмотренных фильмов. Их можно было бы учитывать и как признак того, что человеку не нравится.

4 Сравнение результатов

Каждый способ выдаёт 10 фильмов. Список сравнивается с более поздними оценками пользователя. Совпадением считается фильм с оценкой 4 или 5. Если проверочной оценки нет, совпадение не засчитывается. Это не доказывает, что фильм не понравился бы человеку.

Precision@10 показывает долю совпадений среди десяти рекомендаций. Если совпали 3 фильма, результат равен 3 / 10 = 0.3.

Recall@10 показывает, какую часть всех доступных фильмов с высокой проверочной оценкой удалось найти. Если таких фильмов было 20, а нашлось 3, результат равен 3 / 20 = 0.15.

NDCG@10 учитывает порядок списка: совпадение на первом месте полезнее совпадения на десятом. Каждое совпадение даёт 1 / log2(i + 1), где i это его место. Сумма делится на результат идеального списка. Все три показателя усредняются по пользователям. Чем они выше, тем лучше результат по этой проверке.

Покрытие показывает, какую долю доступных фильмов программа хотя бы раз предложила всей группе. Так можно увидеть разнообразие

общих рекомендаций.

Результаты для всех 287 пользователей:

Способ	Precision@10	Recall@10	NDCG@10	Покрытие
По жанрам	0.2199	0.0629	0.2392	12.13%
По пользователям	0.2467	0.0763	0.2699	12.75%
Популярность	0.2474	0.0859	0.2739	4.58%

У content и collaborative популярность включилась для 193 пользователей: у 192 нет истории, а у ещё одного не оказалось подходящих высоких оценок. Поэтому общий результат сильно зависит от запасного способа. Отдельно рассмотрены 95 пользователей с историей. У одного из них переключение на популярность всё равно сохраняется.

Результаты для 95 пользователей с историей:

Способ	Precision@10	Recall@10	NDCG@10
По жанрам	0.0442	0.0383	0.0545
По пользователям	0.1253	0.0787	0.1472
Популярность	0.1274	0.1078	0.1596

В обоих сравнениях популярность получила лучшие показатели качества. При этом способ по пользователям предложил 206 разных фильмов, а популярность только 74. Способ по жанрам оказался слабее. Один запуск на одном разделении данных не доказывает, что популярность будет лучшей в любом сервисе.

5 Проверка запуска

Проверка выполнена в Windows с Python 3.12.2, NumPy 2.0.2 и scikit-learn 1.6.1. Все 10 тестов прошли за 0.60 с. Они проверяют исключение оценённых и недоступных фильмов, порядок списка, работу с новым пользователем, разделение данных по времени, расчёт показателей и ошибку при неизвестном способе.

Для пользователя 1 content первым предложил As Good As It Gets (1997). У первых десяти фильмов одинаковые жанры Comedy и Drama,

поэтому их числа совпали: 0.7435. Collaborative первым предложил Schindler's List (1993), a popular предложил English Patient, The (1996). Пользователь 1 взят из MovieLens, это не личные оценки автора работы.

У пользователя 0 истории нет. При запуске collaborative появилось «Резерв по популярности: True», первым фильмом стал Star Wars (1977). Значит, запасной способ сработал.

Подготовка данных для рекомендаций заняла около 0.075 с. Среднее время составления списка: по жанрам 0.170 мс, по пользователям 0.058 мс, по популярности 0.045 мс. Это работа с уже загруженными данными. Запуск Python, чтение файлов и сохранение JSON сюда не входят. Скорость при большом числе запросов не проверялась.

6 Недостатки и улучшения

Новый пользователь. Пока ему предлагаются популярные фильмы. Можно попросить выбрать любимые жанры или несколько фильмов. Это поможет составить личный список, но длинный опрос может оттолкнуть человека. Поэтому его лучше сделать необязательным.

Новый фильм. Фильм без ранних оценок пока не попадает в доступный список. Его можно предложить по жанрам или добавить несколько мест для новинок. Но часть таких предложений может оказаться менее подходящей. Это нужно проверить отдельно.

Мало оценок. Люди могут совпасть всего по одному фильму и случайно оказаться похожими. Можно требовать несколько общих понравившихся фильмов. Сравнение станет надёжнее, но у части пользователей не останется подходящих соседей.

Однообразие. Жанровый способ часто предлагает похожие фильмы, а популярность показывает известные названия. Можно добавить больше разных жанров и менее известных фильмов. Однако совпадений с высокими оценками может стать меньше. Ещё один вариант: объединить жанры и выбор похожих пользователей. Настройки такого способа нужно подбирать на отдельных данных, а не на текущей проверке.

Личные данные и нежелательные предложения. История просмотров может раскрывать интересы человека. В настоящем сервисе нужно ограничить доступ к ней, хранить только нужные сведения, учитывать возрастные ограничения и дать возможность скрыть фильм. Фильтры могут ошибаться, поэтому они требуют проверки. В текущем приложении такая защита не сделана.

Накрутка. Фальшивые аккаунты могут продвигать фильм. Можно проверять резкий рост оценок и ограничивать слишком частые действия. Но ограничения иногда мешают честным пользователям. Защита от накрутки в работе не проверялась.

7 Работа в настоящем сервисе

При большом числе пользователей сравнение всех со всеми потребует много памяти. Если пользователей станет вдвое больше, таблица их сходства станет примерно в четыре раза больше. Можно заранее считать соседей и сохранять готовые списки. Это ускорит выдачу, но после новой оценки список обновится не сразу. При выдаче всё равно нужно убрать уже просмотренные и недоступные фильмы.

При обновлении стоит сохранять предыдущую исправную версию. Если новая версия даёт ошибки, можно вернуться к старой. Если личный список не строится, остаётся популярность. Повторно записанные оценки нужно удалять, чтобы они не учитывались несколько раз.

Нужно следить за временем ответа, ошибками, пустыми списками и частотой переключения на популярность. Если переключений стало больше, стоит проверить поступление новых оценок. Также нужно смотреть, обновляются ли данные и не стали ли списки слишком однообразными. Допустимое время ответа можно выбрать после проверки при ожидаемом числе пользователей. Такой проверки пока нет.

8 Польза для сервиса

Важно, чтобы человек быстрее находил интересный фильм и возвращался снова. Одних совпадений с оценками для этого недостаточно. Можно считать долю кликов по рекомендациям, долю начатых и законченных просмотров, время просмотра, возврат через неделю и число скрытых рекомендаций.

Доля кликов называется CTR: число кликов делится на число показов рекомендаций. Но высокий CTR ещё не означает успех. Человек может нажать на фильм из-за названия и сразу закрыть его. Поэтому ожидаемый рост просмотров может не произойти, даже если кликов стало больше.

Для сравнения способов можно случайно разделить пользователей на две постоянные группы. Одной показывать популярные фильмы, другой личные рекомендации. Такой эксперимент называется A/B-тестом. Перед началом нужно выбрать основной показатель и срок проверки. После этого сравнить группы, также учитывая ошибки и время ответа.

В MovieLens нет показов рекомендаций, кликов и времени просмотра. Поэтому эти показатели здесь не рассчитаны. Проверка на сайте и A/B-тест описаны как следующий шаг, они не проводились.

9 Вывод

В программе работают два классических способа и один простой. Все 10 тестов прошли. Для нового пользователя правильно включается популярность.

В этой проверке популярность дала лучшие показатели качества. Способ по пользователям предложил больше разных фильмов, но по точности не обогнал популярность. Способ по жанрам требует более подробных сведений о фильмах. Дальше можно проверить другие периоды данных, улучшить работу с новыми пользователями и фильмами, затем оценить скорость и пользу в настоящем сервисе.

Источники

1. GroupLens. MovieLens 100K. https://grouplens.org/datasets/movielens/100k/

2. GroupLens. Описание и условия использования данных. https://files.grouplens.org/datasets/movielens/ml-100k-README.txt

3. Scikit-learn. TfidfVectorizer. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

4. Scikit-learn. Cosine similarity. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html